Design Document (DD)

OceanSense: An IoT-Based Hybrid System for Real-Time Detection and Tracking of Marine Debris

Project ID : 25-26J-002

Abeysinghe P.T.G.S.R. - IT22253576

Supervisor - Ms. Dinithi Pandithage

BSc (Hons) in Information Technology Specializing in Computer Systems & Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

11th January of 2026

Contents

1. Introduction

1.1 Purpose

The purpose of this Design Document is to provide a detailed and structured specification of the **Swarm Coordination component** of the proposed embedded system. This document defines the architectural design decisions, operational constraints, and system-level requirements necessary to implement decentralized swarm coordination in a real-world embedded and IoT environment. The document serves as a reference for system designers, developers, evaluators, and maintainers to ensure consistent understanding and correct implementation of the swarm coordination mechanism.

1.2 Scope

This document section focuses exclusively on the Swarm Coordination subsystem, which is responsible for enabling multiple autonomous embedded nodes to operate collaboratively without centralized control. The scope includes swarm architecture, coordination logic, communication mechanisms, performance requirements, constraints, and limitations. Low-level hardware schematics and detailed firmware code are outside the scope of this document and are addressed in implementation-level artifacts.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
Node	An autonomous embedded unit participating in the swarm
IoT	Internet of Things
Decentralized Control	Control strategy without a single central controller
Local Interaction	Decision-making based on neighboring node information
Emergent Behavior	Global behavior resulting from local interactions

1.4 Overview

The Swarm Coordination component enables multiple embedded nodes to collaboratively perform tasks through local decision-making and node-to-node communication. Each node operates independently while continuously interacting with neighboring nodes, allowing the swarm to adapt dynamically to environmental changes, node failures, and scalability requirements. This approach improves robustness, fault tolerance, and flexibility compared to traditional centralized coordination systems.

2. Overall Description

2.1 Product Perspective

The Swarm Coordination subsystem is a core functional component of the proposed embedded system and operates as a distributed control layer across multiple autonomous nodes. Unlike conventional centralized or leader–follower coordination architectures, the proposed system adopts a decentralized approach in which each node independently executes coordination logic based on local sensing and node-to-node communication. This design eliminates single points of failure and enhances scalability, adaptability, and robustness in dynamic environments.

From a system perspective, the swarm coordination module integrates with sensing, communication, and actuation subsystems on each embedded node. Collective swarm behavior emerges from continuous local interactions.

2.1.1 System Interfaces

The swarm coordination logic interfaces with the embedded operating system, sensor drivers, and communication stacks on each node.

2.1.2 User Interfaces

The system does not require direct user interaction for swarm operation. An optional supervisory interface may be provided to visualize node status, swarm topology, and operational metrics for monitoring and evaluation purposes only.

2.1.3 Hardware Interfaces

Each swarm node interfaces with onboard sensors such as GPS, IMU , actuators (motors), and embedded controllers. These interfaces provide real-time environmental data and enable physical actuation based on coordination decisions.

2.1.4 Software Interfaces

The subsystem interfaces with supporting firmware, sensor processing modules, and motor control modules.

2.1.5 Communication Interfaces

Inter-node communication is achieved through wireless communication technology LoRa. The communication interface supports periodic state broadcasting, event-driven messaging, and neighbor discovery mechanisms.

2.1.6 Memory Constraints

The swarm coordination logic is designed to operate within the limited memory resources of embedded platforms. Memory usage is optimized by minimizing message payload sizes and avoiding centralized data storage.

2.1.7 Operations

Normal system operation includes node initialization, neighbor discovery, periodic state exchange, local decision-making, adaptive movement or task execution, and dynamic reconfiguration in response to node failure or environmental changes.

State switching in the swarm coordination algorithm is **rule-based and priority-driven**.

Each ASV continuously evaluates its **local environment and neighbor conditions** at every control step. Based on these inputs, it selects **one operational state**, also referred to as a behavior mode.

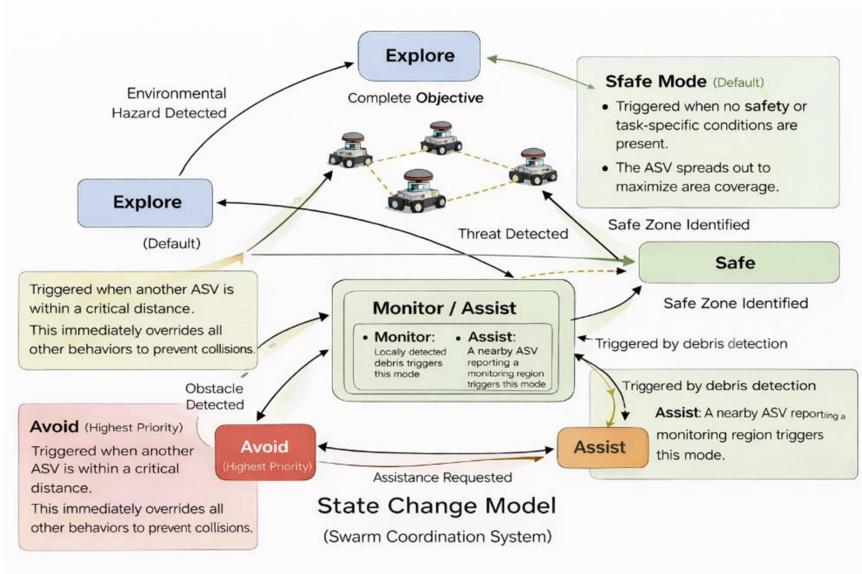
Environment Conditions sense by each ASV :

- Sea Current
- Sea Wave
- Visibility

The default state is **Explore**, which focuses on area coverage.
When specific conditions are detected, the ASV switches to a higher-priority state.

The switching follows a **strict priority hierarchy**:

1. **Avoid Mode (Highest Priority)**
2. **Safe Mode**
3. **Monitor Mode**
4. **Assist Mode**
5. **Explore Mode (Default)**



2.1.8 Site Adaptation Requirements

Swarm nodes must be deployed within wireless communication range of at least one neighboring node. Environmental factors such as obstacles, interference, and terrain must be considered when placing nodes.

Sensors and antennas should be positioned to minimize obstruction and maximize sensing and communication reliability.

2.2 Product Functions

The primary functions of the Swarm Coordination subsystem include:

- Autonomous coordination among multiple nodes
- Decentralized decision-making based on local information
- Dynamic adaptation to changes in swarm size and topology
- Distributed task allocation and movement control
- Local sensing and neighbor-based decision-making

2.3 User Characteristics

The primary users of the Swarm Coordination system are researchers, students, and engineers involved in embedded systems, robotics, or IoT development. Users are expected to have basic technical knowledge related to embedded devices and networking.

No advanced user interaction is required during normal system operation. Secondary users may include system maintainers responsible for configuration and monitoring.

2.4 Constraints

- Limited communication bandwidth

- Power and battery limitations
- Environmental interference affecting wireless communication
- Limited processing power and memory of embedded nodes

2.5 Assumptions and Dependencies

The system assumes that each node maintains communication with at least one neighboring node during operation. It is also assumed that sensor data accuracy remains within acceptable bounds and that basic time synchronization is available across nodes. Swarm nodes are powered continuously during operation.

2.6 Apportioning of Requirements

The implementation order of system requirements is as follows:

- Deployment and initialization of swarm nodes
- Implementation of neighbor discovery and communication
- Integration of local sensing and coordination logic
- Execution of cooperative swarm behaviors
- Optimization and scalability enhancements

3. Specific Requirements (Embedded System Oriented – DD)

3.1 External Interface Requirements

3.1.1 User Interfaces

The Swarm Coordination system is designed to operate autonomously without continuous human control. Therefore, no direct operational user interface is required for normal swarm functioning.

An web dashboard monitoring interface provided for observation and evaluation purposes. This interface shall display:

- Swarm node identifiers
- Node connectivity status
- Current swarm formation or distribution
- Basic status indicators such as active, inactive, or disconnected nodes

The user interface shall be intuitive and minimalistic, intended for users with basic technical knowledge. The interface shall not interfere with autonomous swarm decision-making.

3.1.2 Hardware Interfaces

The Swarm Coordination system shall interface with the following hardware components:

- **Swarm Nodes:** Embedded controller ESP32 with Raspberry Pi responsible for local decision-making.
- **Sensors:** GPS modules, Compass or Gyro sensors used for local environment perception.
- **Actuators:** Motors or propulsion systems enabling physical movement or task execution
- **Power Units:** Battery or regulated power supplies for continuous operation

Each node independently interfaces with its sensors and actuators to support decentralized operation.

3.1.3 Software Interfaces

The Swarm Coordination subsystem shall interface with the following software components:

- Embedded firmware responsible for sensor data acquisition
- Local decision-making and coordination logic
- Wireless communication protocol stacks
- Motor control and actuation software modules

All software interfaces shall follow a modular structure to ensure separation between sensing, communication, coordination, and actuation functionalities.

3.1.4 Communication Interfaces

Swarm nodes shall communicate using decentralized wireless communication mechanisms. Supported communication method is Lora and additionally we have Wi-Fi on embedded controller

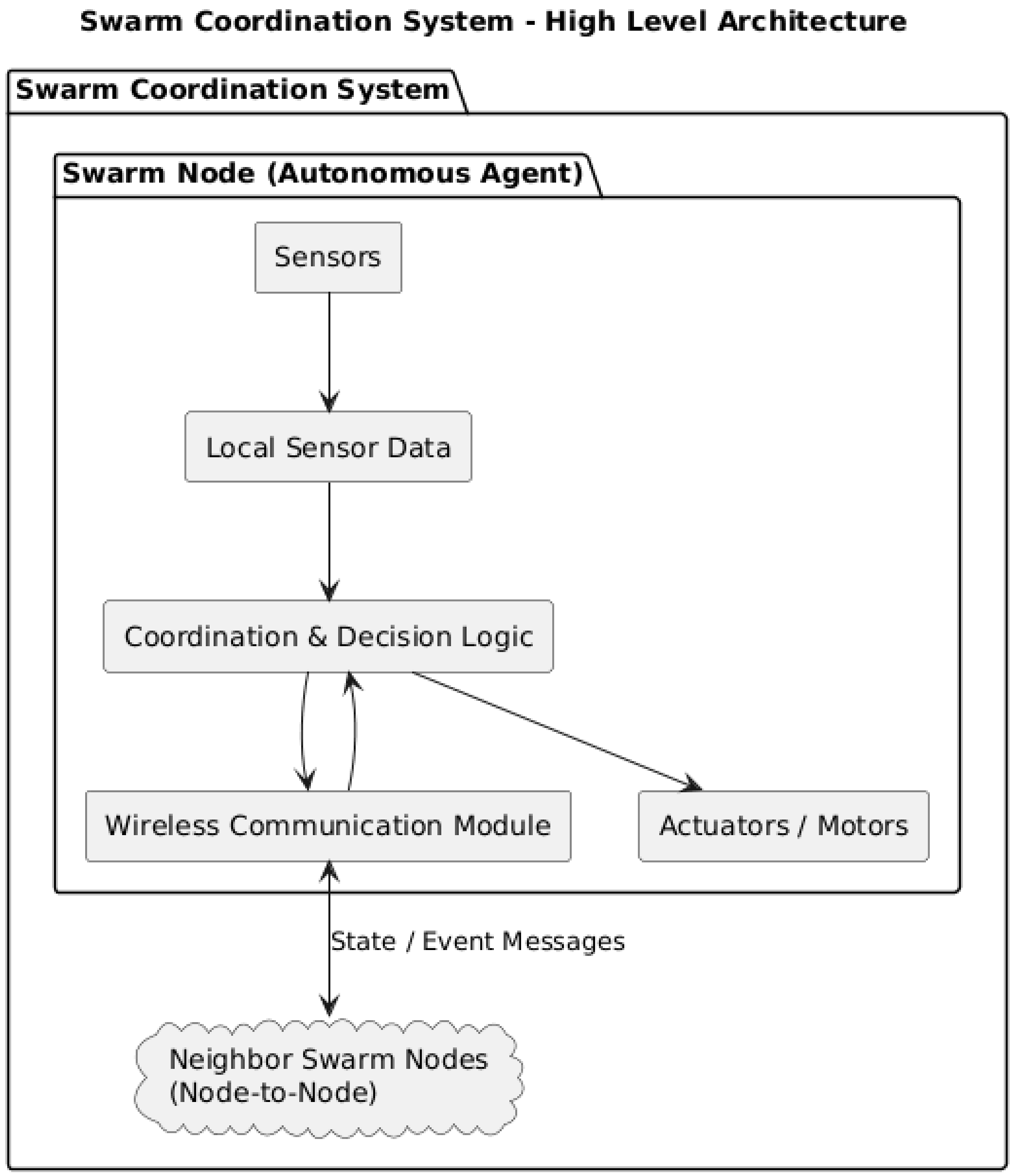
The communication interface shall support:

- Neighbor discovery
- Periodic state information exchange
- Event-based coordination messages

No permanent centralized communication controller shall be required.

3.2 Architectural Design

3.2.1 High-Level Architectural Design



Each swarm node operates as an autonomous agent executing local coordination logic. Nodes exchange minimal state information with neighbors, enabling collective swarm behavior without centralized supervision.

3.2.2 Hardware and Software Requirements with Justification

Low-power microcontrollers and single-board computers are selected to balance computational capability with energy efficiency.

Component	Requirement	Justification
ESP32 / Raspberry Pi	Adequate RAM and Flash	Supports local coordination logic and communication
Wireless Module	Wi-Fi / LoRa	Enables decentralized node-to-node communication
Sensors	Digital / Analog	Provides local environmental awareness
Motors / Actuators	Low-power drivers	Enables movement and task execution

The use of low-power embedded platforms ensures scalability and energy efficiency. Modular software design allows easy extension and modification of swarm behaviors.

3.2.3 Risk Mitigation Plan

Risk	Mitigation Strategy
Node failure	Swarm dynamically reconfigures using remaining nodes
Communication loss	Nodes rely on local sensing and last known neighbor states
Network congestion	Adaptive communication frequency
Power depletion	Energy-aware coordination or node withdrawal

As an alternative solution, temporary leader election may be used during extreme communication failures.

3.2.4 Cost–Benefit Analysis

Decentralized coordination reduces infrastructure cost and increases robustness, outweighing the additional complexity of distributed logic.

Component	Cost (Approx.)	Benefit
Embedded Swarm Nodes	Moderate	Autonomous decentralized operation
Wireless Communication	Low	Scalable and infrastructure-free coordination
Sensors & Actuators	Moderate	Enables intelligent collective behavior

3.3 Performance Requirements

- Each swarm node shall process sensor data and coordination logic within acceptable real-time limits.
- Communication latency between neighboring nodes shall support timely coordination decisions.
- The system shall support dynamic scaling of swarm size without major performance degradation.
- Memory and CPU usage shall remain within embedded system constraints.

3.4 Design Constraints

- Embedded memory and processing limitations
- Wireless communication interference and packet loss
- Power and battery capacity limitations
- Environmental uncertainty affecting sensor accuracy

3.5 Software System Attributes

3.5.1 Reliability

The swarm shall continue operating despite individual node failures, provided a minimum number of nodes remain active.

3.5.2 Availability

The swarm coordination functionality shall remain available as long as sufficient nodes are powered and within communication range.

3.5.3 Security

Basic security mechanisms shall ensure message integrity and prevent unauthorized nodes from joining the swarm.

3.5.4 Maintainability

The modular architecture shall allow easy updates to coordination logic, communication strategies, and behavioral parameters.

3.6 Other Requirements

Future enhancements may include learning-based coordination and adaptive behavior tuning.

- Learning-based swarm coordination
- Adaptive behavior tuning
- Energy-aware swarm optimization

- Add debris collecting mechanism with monitoring
- Alerting system

4. Limitations

- Communication delays may affect coordination accuracy
- Battery constraints limit long-term operation
- Performance degrades under high node density

5. Supporting Information

5.1 Appendices

Explore mode grid cell calculation and assignment(Swarm_Controller.py)

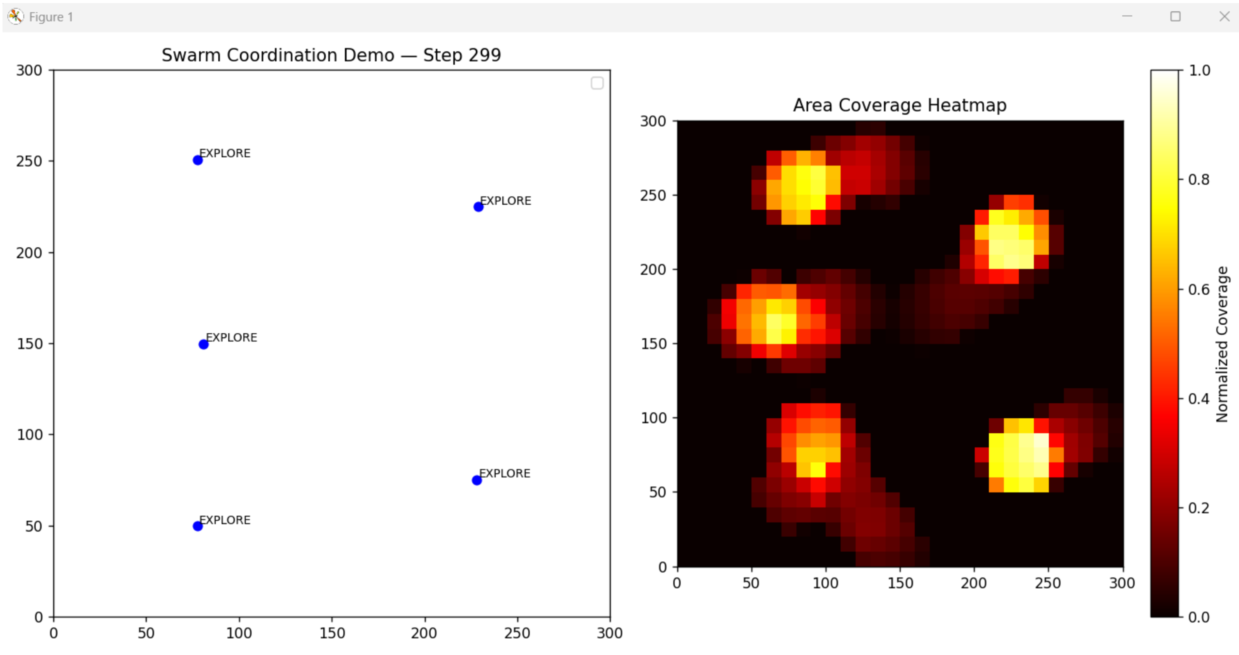
```python
# -------------------------------------------------
# VORONOI HELPERS (EXPLORE)
# -------------------------------------------------
def assign_voronoi_cells(self, nodes, grid_centers):
    """
    Assign each grid cell to the nearest node.
    Returns dict: {node_id: [cell_centers]}
    """
    voronoi_map = {node.id: [] for node in nodes}

    for cell in grid_centers:
        min_dist = float("inf")
        closest_id = None
        for node in nodes:
            dist = np.linalg.norm(cell - node.position)
            if dist < min_dist:
                min_dist = dist
                closest_id = node.id
        voronoi_map[closest_id].append(cell)

    return voronoi_map
```

```python
# --------------------------------
# EXPLORE MODE (LLOYD CONTROL)
# --------------------------------
if node.mode == "EXPLORE" and my_voronoi_cells:
    centroid = np.mean(my_voronoi_cells, axis=0)
    direction = centroid - node.position
    dist = np.linalg.norm(direction)
    if dist > 0:
        movement += direction / dist
```

Simulation Output



5.2 References

[1] M. Dorigo, M. Birattari, and M. Brambilla, “Swarm Robotics,” *Scholarpedia*, vol. 9, no. 1, pp. 1463–1478, 2014.

[2] A. Brambilla, M. Birattari, D. M. Mateos, and M. Dorigo, “Swarm robotics: A review from the swarm engineering perspective,” *Swarm Intelligence*, vol. 7, no. 1, pp. 1–41, 2013.

[3] Y. Tan and Z. Zheng, “Research Advance in Swarm Robotics,” *Defence Technology*, vol. 9, no. 1, pp. 18–39, 2013.

[4] J. Chen and S. Li, “Decentralized Control of Multi-Agent Systems: A Review,” *IEEE Transactions on Control Systems Technology*, vol. 25, no. 5, pp. 1609–1622, Sep. 2017.

[5] L. Bayındır, “A review of swarm robotics tasks,” *Neurocomputing*, vol. 172, pp. 292–321, Jan. 2016.

[6] F. Bullo, J. Cortés, and S. Martínez, *Distributed Control of Robotic Networks*, Princeton University Press, 2009.

[7] J. P. Queralta et al., “Collaborative Multi-Robot Systems for Marine and Aerial Monitoring: A Survey,” *IEEE Access*, vol. 8, pp. 140–155, 2020.

[8] A. V. Savkin and H. Huang, “Navigation of Autonomous Vehicles for Maritime Surveillance and Environmental Monitoring,” *Robotics and Autonomous Systems*, vol. 59, no. 7–8, pp. 574–585, 2011.